

GRADO EN INGENIERÍA INFORMÁTICA



VNIVERSITAT
ID VALÈNCIA

TRABAJO FIN DE GRADO

QSIMOV: RE-ENTRENAMIENTO INCREMENTAL
SIN OLVIDO CATASTRÓFICO EN REDES
NEURONALES CON ACTIVACIÓN RELU

AUTOR:

MATÍAS MONCHO RONDA

TUTOR:

**DR. JUAN MANUEL ORDUÑA
HUERTAS**



VNIVERSITAT
DE VALÈNCIA



Escola Tècnica Superior
d'Enginyeria **ETSE-UV**

TRABAJO FIN DE GRADO

QSIMOV: RE-ENTRENAMIENTO INCREMENTAL SIN OLVIDO CATASTRÓFICO EN REDES NEURONALES CON ACTIVACIÓN RELU

AUTOR: MATÍAS MONCHO RONDA

**TUTOR: DR. JUAN MANUEL ORDUÑA
HUERTAS**

Declaración Responsable de Autoría y Uso de Inteligencia Artificial (IA) en el TFG

ETSE-UV

Herramientas de IA Utilizadas

Declaro que en este trabajo he utilizado las siguientes herramientas de IA (si procede):

1. Herramienta y versión:
2. Herramienta y versión:

1) Dimensiones de Uso de la IA

Instrucciones: Determina el grado de uso de la IA en las siguientes dimensiones.

Marca una opción por fila.

Escala (5 puntos): 1 = No se ha utilizado IA, 2 = Uso muy puntual, 3 = Uso moderado, 4 = Uso frecuente, 5 = Uso intensivo

Dimensión	1	2	3	4	5
Corrección lingüística: ortografía, gramática, puntuación y coherencia formal del texto					
Reescritura y mejora de estilo: reformulación para aumentar claridad, concisión y legibilidad manteniendo el significado					
Síntesis de contenido: elaboración de resúmenes a partir de textos extensos o complejos para identificar ideas clave					
Apoyo en la localización de referencias: propuesta de posibles fuentes (a verificar y citar correctamente)					
Planificación del trabajo: propuesta de esquemas, índices o guiones iniciales para estructurar el documento					
Mejora de la argumentación y la estructura: detección de huecos, redundancias o saltos lógicos y propuesta de reorganización					
Programación (si procede): apoyo en el desarrollo, depuración u optimización de fragmentos de código puntuales					
Programación (si procede): uso generalizado de herramientas de generación de código de forma autónoma a partir de especificaciones					
Análisis de datos (si procede): ayuda exploratoria para limpieza preliminar, generación de hipótesis e identificación de patrones					
Traducción o adaptación lingüística (si procede)					

Dimensión	1	2	3	4	5
Soporte en material visual (si procede): ideas para figuras, tablas o gráficos y revisión de su explicación					
Otros (especificar si procede)					

2) Compromisos de Uso Responsable

Instrucciones: Determina si se ha realizado un uso responsable de la IA. Marca tu grado de acuerdo.

Escala (acuerdo): 1 = Totalmente en desacuerdo, 2 = En desacuerdo, 3 = Neutral, 4 = De acuerdo, 5 = Totalmente de acuerdo

Declaración	1	2	3	4	5
Las ideas principales, interpretaciones y análisis críticos del trabajo son propios.					
El uso de IA no ha sustituido mis competencias básicas de investigación, redacción y razonamiento académico.					
He revisado, contrastado e integrado críticamente cualquier contenido generado con IA antes de incorporarlo.					
He verificado la precisión y fiabilidad del contenido, incluyendo datos, afirmaciones y referencias bibliográficas.					

Observaciones (Opcional)

Declaración Final

A través de esta declaración, afirmo que he descrito de manera transparente y fidedigna el uso de IA de este trabajo. Asumo la autoría y responsabilidad de la obra final y confirmo que no he introducido datos personales o confidenciales contraviniendo la normativa de protección de datos personales¹ y que he respetado las normas en materia de propiedad intelectual², así como las previsiones sobre integridad académica³ de la Universitat de València.

Firma y Autorización

Fecha: _____

Firma:

¹Normativa de protección de datos personales de la Universitat de València

²Normativa de propiedad intelectual de la Universitat de València

³Normativa sobre integridad académica de la Universitat de València

Resumen:

El aprendizaje continuo en redes neuronales artificiales choca con un problema estructural conocido como olvido catastrófico: cuando una red es re-entrenada con nuevas muestras, el ajuste de sus pesos destruye el conocimiento previamente adquirido. Este trabajo evalúa **Qsimov**, una librería Python existente que aborda este problema mediante la explotación de la estructura lineal por tramos que inducen las activaciones ReLU en redes feed-forward.

La idea central consiste en representar el comportamiento de las últimas capas de una red pre-entrenada en términos de sus **camino activos**: las combinaciones de conexiones entre neuronas que permanecen activas para cada muestra dada. Esta representación transforma el re-entrenamiento en la resolución de un sistema de ecuaciones lineales cuyas incógnitas son los pesos de los caminos, y no los pesos individuales de la red original. Gracias a esta reformulación, es posible acumular las ecuaciones de distintas rondas de entrenamiento sin necesidad de acceder a los datos anteriores, eliminando así el olvido. La librería ofrece dos mecanismos de re-entrenamiento: resolución directa mediante el sistema lineal (variante más rápida) y descenso de gradiente sobre una capa completamente conectada restringida a los caminos activos (variante más flexible). Ambos se implementan sobre TensorFlow/Keras y PyTorch con extensiones en Cython para las operaciones más costosas.

Los experimentos, realizados sobre MNIST, CIFAR-10 e ImageNet con 100 clases, demuestran que el sistema lineal con acumulación de ecuaciones elimina el olvido catastrófico en todos los escenarios evaluados: mantiene 94.3 % de precisión en clases antiguas en MNIST (frente al 0.0 % del fine-tuning estándar), 73.4 % en CIFAR-10, y entre el 79 % y el 84 % en rondas previas sobre ImageNet, todo ello entre seis y diez veces más rápido que el re-entrenamiento acumulado estándar.

Abstract:

Continual learning in artificial neural networks is hindered by a structural problem known as catastrophic forgetting: when a network is retrained on new samples, the weight updates destroy previously acquired knowledge. This work evaluates **Qsimov**, an existing Python library that addresses this problem by exploiting the piecewise-linear structure induced by ReLU activations in feed-forward networks.

The core idea is to represent the behaviour of the final layers of a pre-trained network in terms of their **active paths**: the combinations of neuron connections that remain active for each given sample. This representation transforms retraining into solving a linear system of equations whose unknowns are the path weights, not the individual weights of the original network. Thanks to this reformulation, equations from different training rounds can be accumulated without requiring access to previous data, thereby eliminating forgetting.

The library offers two retraining mechanisms: direct solving via the linear system (faster variant) and gradient descent over a fully connected layer restricted to the active paths (more flexible variant). Both are implemented for TensorFlow/Keras and PyTorch, with Cython extensions for the most computationally intensive operations.

Experiments conducted on MNIST, CIFAR-10 and ImageNet with 100 classes demonstrate that the linear system with equation accumulation eliminates catastrophic forgetting in all evaluated scenarios: it maintains 94.3 % accuracy on old classes in MNIST (versus 0.0 % for standard fine-tuning), 73.4 % in CIFAR-10, and between 79 % and 84 % in previous rounds on ImageNet, all between six and ten times faster than standard cumulative retraining.

Resum:

L'aprenentatge continu en xarxes neuronals artificials es troba amb un problema estructural conegut com a oblit catastròfic: quan una xarxa és re-entrenada amb noves mostres, l'ajust dels seus pesos destrueix el coneixement prèviament adquirit. Aquest treball avalua **Qsimov**, una llibreria Python existent que aborda aquest problema mitjançant l'explotació de l'estructura lineal a trams que indueixen les activacions ReLU en xarxes feed-forward.

La idea central consisteix a representar el comportament de les últimes capes d'una xarxa pre-entrenada en termes dels seus **camins actius**: les combinacions de connexions entre neurones que romanen actives per a cada mostra donada. Aquesta representació transforma el re-entrenament en la resolució d'un sistema d'equacions lineals les incògnites de les quals són els pesos dels camins, i no els pesos individuals de la xarxa original. Gràcies a aquesta reformulació, és possible acumular les equacions de distintes rondes d'entrenament sense necessitat d'accedir a les dades anteriors, eliminant així l'oblit.

La llibreria ofereix dos mecanismes de re-entrenament: resolució directa mitjançant el sistema lineal (variant més ràpida) i descens del gradient sobre una capa completament connectada restringida als camins actius (variant més flexible). Tots dos s'implementen sobre TensorFlow/Keras i PyTorch amb extensions en Cython per a les operacions més costoses.

Els experiments, realitzats sobre MNIST, CIFAR-10 i ImageNet amb 100 classes, demostren que el sistema lineal amb acumulació d'equacions elimina l'oblit catastròfic en tots els escenaris avaluats: manté un 94.3 % de precisió en classes antigues en MNIST (enfront del 0.0 % del fine-tuning estàndard), un 73.4 % en CIFAR-10, i entre el 79 % i el 84 % en rondes prèvies sobre ImageNet, tot plegat entre sis i deu vegades més ràpid que el re-entrenament acumulat estàndard.

Índice general

1. Introducción y Motivación	19
1.1. Introducción	19
1.2. Motivación y Objetivos	20
1.2.1. Motivación	20
1.2.2. Objetivos	21
2. Estado del Arte	23
2.1. El problema del olvido catastrófico	23
2.2. Taxonomía de soluciones	23
2.2.1. Métodos basados en regularización	23
2.2.2. Métodos basados en repetición (<i>replay</i>)	24
2.2.3. Métodos basados en arquitectura	24
2.3. Linealidad local de redes ReLU	25
2.4. Sistemas lineales para re-entrenamiento	25
2.5. Resumen y posicionamiento de Qsimov	25
3. Especificación del Sistema	27
3.1. Definición de requisitos	27
3.1.1. Requisitos funcionales	27
3.1.2. Requisitos no funcionales	28
3.2. Especificación del sistema	28
3.3. Planificación y estimación de costes	28
3.3.1. Planificación	28
3.3.2. Estimación de costes	28

4. Desarrollo del Proyecto	31
4.1. Análisis del sistema	31
4.1.1. Formalización matemática de los caminos	31
4.1.2. Casos de uso principales	32
4.1.3. Modelado de datos	33
4.2. Diseño del sistema	33
4.2.1. Diseño de interfaces de usuario	33
4.2.2. Diseño de procesos: extracción de caminos	34
4.2.3. Diseño de la base de datos (persistencia)	34
4.3. Arquitectura interna de Qsimov	35
4.3.1. Módulo <code>paths/</code> : Generación de caminos (Python + Cython)	35
4.3.2. Módulo <code>linalg.py</code> : Resolución del sistema	35
4.3.3. Diferencias de implementación entre Keras y PyTorch	36
5. Pruebas y Resultados	37
5.1. Descripción de experimentos	37
5.1.1. Configuración experimental común	37
5.1.2. Experimento 1: Velocidad y precisión (MNIST)	38
5.1.3. Experimento 2: Robustez al learning rate (MNIST)	39
5.1.4. Experimento 3: Olvido catastrófico (MNIST)	40
5.1.5. Experimento 4: CIFAR-10. Olvido catastrófico y datos escasos	41
5.1.6. Experimento 5: ImageNet-100	42
5.2. Resultados y discusión	45
5.3. Evaluación presupuestaria	46
6. Conclusiones y Trabajo Futuro	47
6.1. Conclusiones	47
6.2. Trabajo futuro	48
A. Anexos	49
A.1. Anexo A: Instalación y configuración del entorno	49

A.2. Anexo B: Ejemplo de uso completo (Keras, aprendizaje continuo)	49
A.3. Anexo C: Guía de reproducibilidad	50
A.3.1. C.1 Entorno de ejecución	50
A.3.2. C.2 Instalación del entorno	50
A.3.3. C.3 Descarga y preprocesado de datos	51
A.3.4. C.4 Comandos de ejecución por experimento	51
A.3.5. C.5 Consulta de resultados	53
A.4. Anexo D: Software desarrollado	54
A.4.1. D.1 Módulo de infraestructura de experimentos	54
A.4.2. D.2 Experimento 1: Velocidad y precisión (MNIST)	54
A.4.3. D.3 Experimento 2: Robustez al learning rate (MNIST)	55
A.4.4. D.4 Experimentos de olvido catastrófico (MNIST y CIFAR-10)	55
A.4.5. D.5 Experimentos de ImageNet	55
Referencias	56

Capítulo 1

Introducción y Motivación

1.1. Introducción

Las redes neuronales profundas dominan actualmente el estado del arte en tareas de visión por computador, procesamiento del lenguaje natural y reconocimiento de patrones. Sin embargo, el paradigma dominante de entrenamiento (procesar todos los datos de una vez con acceso irrestricto) resulta inapropiado en muchos escenarios reales. Los datos llegan de forma secuencial, los recursos computacionales son limitados, y las restricciones de privacidad o almacenamiento impiden conservar todos los datos históricos. En este contexto surge el campo del **aprendizaje continuo** (*continual learning*), cuyo objetivo es que un modelo aprenda de nuevos datos sin olvidar los conocimientos previos.

El obstáculo principal de este campo es el **olvido catastrófico** (*catastrophic forgetting*), descrito por primera vez de forma sistemática por McCloskey y Cohen en 1989 [1]. Cuando una red neuronal es re-entrenada sobre nuevas muestras mediante descenso de gradiente, la actualización de sus pesos interfiere destructivamente con las representaciones aprendidas anteriormente, de forma mucho más severa que el olvido observado en sistemas biológicos.

La mayoría de las soluciones propuestas en la literatura (regularización, expansión de arquitectura, o replay de datos generados) introducen compromisos entre plasticidad (capacidad de aprender cosas nuevas) y estabilidad (capacidad de retener las anteriores), y frecuentemente requieren almacenar ejemplos pasados o aumentar la complejidad del modelo con cada nueva tarea.

Este trabajo evalúa Qsimov, una librería que implementa un enfoque alternativo basado en una propiedad matemática de las redes con activación ReLU: para cualquier entrada, la función que computa la red es **lineal** en

la región del espacio que contiene a esa entrada. Cada entrada traza un «camino» único a través de la red, determinado por qué neuronas están activas (> 0) y cuáles no. Qsimov separa la estructura de esos caminos, aprendida durante el pre-entrenamiento, de los pesos asociados a cada camino, que se resuelven analíticamente mediante un sistema de ecuaciones lineales. La acumulación incremental de ecuaciones a lo largo del tiempo garantiza que el sistema «recuerde» todas las rondas de entrenamiento sin necesidad de acceder a los datos originales.

El presente documento describe el funcionamiento de Qsimov y los experimentos realizados para evaluar su comportamiento en escenarios de aprendizaje continuo, siguiendo la estructura recomendada para trabajos de fin de grado de ingeniería en informática [2].

1.2. Motivación y Objetivos

1.2.1. Motivación

La motivación del proyecto nace de una observación fundamental sobre la naturaleza del entrenamiento de redes neuronales con activación ReLU. Una red feed-forward con ReLU define, implícitamente, una partición del espacio de entrada en regiones lineales: dentro de cada región, la función que computa la red es una transformación lineal afín [3]. Esa transformación queda completamente determinada por el subconjunto de neuronas que están activas para las entradas de esa región, es decir, por el «camino activo» de la muestra.

Esta propiedad tiene una consecuencia directa para el re-entrenamiento: si se fijan los parámetros que determinan *qué* caminos existen (los pesos del modelo base, aprendidos durante el pre-entrenamiento), el problema de encontrar los pesos óptimos para esos caminos se reduce a resolver un sistema de ecuaciones lineales. Y un sistema lineal puede ampliarse con nuevas ecuaciones en cualquier momento, incorporando nuevas muestras o nuevas clases, sin que las soluciones previas se destruyan, siempre que las ecuaciones antiguas permanezcan en el sistema.

Frente a las soluciones existentes basadas en penalizaciones (EWC [4], SI [5]) o en expansión de arquitectura (PNN [6]), Qsimov ofrece garantías exactas de no-olvido para las muestras incluidas en el sistema, sin requerir hiperparámetros adicionales y sin aumentar el tamaño de la red.

1.2.2. Objetivos

Los objetivos del proyecto son los siguientes:

El objetivo principal del proyecto es evaluar experimentalmente la librería Qsimov en escenarios de aprendizaje continuo, caracterizando su comportamiento frente al olvido catastrófico y comparándolo con métodos de re-entrenamiento estándar.

Los objetivos secundarios son:

1. Comprender el funcionamiento de los componentes principales de Qsimov (`PathSelector`, `QsimovLinearSystem`, `QsimovGradient`) y su API de uso.
2. Diseñar y ejecutar experimentos de evaluación sobre tres *benchmarks* de distinta escala y complejidad: MNIST, CIFAR-10 e ImageNet (100 clases).
3. Comparar el rendimiento de Qsimov (variante de sistema lineal y variante de gradiente) con métodos de referencia: fine-tuning estándar y re-entrenamiento acumulado.
4. Analizar el impacto del punto de corte (`initial_layer`) sobre la precisión obtenida y el número de caminos generados.
5. Evaluar la robustez al learning rate del módulo de gradiente frente al re-entrenamiento estándar.
6. Identificar los escenarios en los que Qsimov ofrece ventajas claras y aquellos en los que presenta limitaciones.

Capítulo 2

Estado del Arte

2.1. El problema del olvido catastrófico

El olvido catastrófico fue caracterizado por primera vez de forma rigurosa por McCloskey y Cohen [1] y Ratcliff [7], que demostraron experimentalmente cómo incluso un entrenamiento mínimo sobre nueva información borraba casi por completo las representaciones previamente aprendidas en modelos conexionistas. A diferencia del olvido humano, que es gradual y selectivo, el olvido en redes artificiales es abrupto y masivo: basta con actualizar los pesos mediante descenso de gradiente sobre un nuevo conjunto de datos para que el rendimiento en los datos anteriores caiga drásticamente.

La raíz del problema reside en la naturaleza distribuida de las representaciones en redes neuronales: los mismos pesos participan en la codificación de múltiples patrones, de modo que su modificación para adaptarse a los nuevos datos necesariamente interfiere con los antiguos. Este conflicto entre capacidad de aprender cosas nuevas y capacidad de retener las ya aprendidas se denomina el **dilema estabilidad-plasticidad** [8].

2.2. Taxonomía de soluciones

Las estrategias para mitigar el olvido catastrófico se agrupan en tres grandes familias [9]:

2.2.1. Métodos basados en regularización

Estos métodos añaden un término de penalización a la función de pérdida para evitar que los pesos más importantes para las tareas anteriores cambien demasiado. El más influyente es **EWC** (*Elastic Weight Consolidation*, Kirk-

patrick et al., 2017) [4], que usa la diagonal de la matriz de información de Fisher como medida de la importancia de cada peso. Formalmente, la función de pérdida para la tarea B es:

$$L_B(\theta) = L_{B_orig}(\theta) + (\lambda/2) * \sum_i F_i * (\theta_i - \theta_{A,i})^2$$

donde θ_A^* son los pesos óptimos para la tarea A, F_i es la importancia del peso i , y λ controla el equilibrio entre plasticidad y estabilidad. **SI** (*Synaptic Intelligence*, Zenke et al., 2017) [5] estima las importancias de forma online durante el entrenamiento, evitando el coste computacional del cálculo de la Fisher.

El principal inconveniente de estos métodos es que la penalización es solo una aproximación: no garantizan la ausencia total de olvido, y la importancia calculada sobre una tarea puede no ser compatible con las importancias calculadas sobre múltiples tareas sucesivas.

2.2.2. Métodos basados en repetición (*replay*)

Estos métodos mantienen una memoria de ejemplos pasados (reales o generados) que se mezclan con los datos nuevos durante el re-entrenamiento. El **replay exacto** almacena directamente subconjuntos de datos históricos (*coreset methods*), con el coste de violar potencialmente restricciones de privacidad o almacenamiento. El **replay generativo** (Shin et al., 2017) [10] usa un modelo generativo (típicamente una GAN) entrenado sobre las tareas pasadas para sintetizar ejemplos artificiales que se mezclan con los datos nuevos.

El replay generativo elimina la necesidad de almacenar datos reales, pero introduce un nuevo modelo que debe entrenarse a su vez de forma continua, trasladando el problema del olvido al generador.

2.2.3. Métodos basados en arquitectura

Estos métodos modifican la estructura de la red para aislar el conocimiento de cada tarea. Las **Redes Neuronales Progresivas** (PNN, Rusu et al., 2016) [6] añaden una columna nueva a la red para cada tarea nueva, con conexiones laterales a las columnas anteriores, pero congelando los pesos previos. Esto garantiza que las tareas antiguas no se olvidan, pero el tamaño del modelo crece linealmente con el número de tareas.

Los métodos basados en máscaras binarias (*PackNet*, Mallya y Lazebnik, 2018 [11]; *HAT*, Serrà et al., 2018 [12]) reservan subconjuntos de pesos para

cada tarea, pero requieren conocer el número de tareas de antemano y tienen una capacidad finita.

2.3. Linealidad local de redes ReLU

Una propiedad matemática fundamental de las redes con activación ReLU es que definen funciones **continuas y lineales a tramos**: para una entrada dada, la composición de transformaciones afines y funciones ReLU resulta en una transformación lineal afín en la región convexa del espacio de entrada que contiene a esa muestra [3]. Esta región queda determinada por el **patrón de activación** binario: el vector que indica qué neuronas tienen salida positiva y cuáles cero.

Esta propiedad ha sido estudiada desde la perspectiva de la capacidad representacional (las redes ReLU profundas pueden generar un número de regiones lineales exponencial con la profundidad [3]) y más recientemente desde la perspectiva del análisis de robustez [13]. Qsimov explota esta propiedad de forma constructiva: en lugar de analizar las regiones lineales de la red completa, extrae y enumera los **camino de conexión activos** entre capas, que son la representación dual de las regiones lineales cuando se consideran las conexiones capa a capa.

2.4. Sistemas lineales para re-entrenamiento

La idea de reformular el entrenamiento de redes neuronales como un problema de mínimos cuadrados es clásica en el contexto de las capas de salida. La técnica **LWTA** (*Local Winner Takes All*, Srivastava et al., 2013 [14]) y los métodos de **extreme learning machines** (ELM, Huang et al., 2006 [15]) fijan los pesos de las capas ocultas aleatoriamente y entrenan solo la última capa con mínimos cuadrados. Qsimov se distingue de estos enfoques en que los pesos del modelo base no son aleatorios sino pre-entrenados, y en que la representación lineal no se aplica solo a la última capa sino a una secuencia de capas cuyo punto de inicio es configurable.

2.5. Resumen y posicionamiento de Qsimov

La tabla siguiente sitúa Qsimov en relación con las principales aproximaciones al aprendizaje continuo:

Tabla 2.1: Comparativa de métodos de aprendizaje continuo.

Método	Sin datos previos	Sin arq. creciente	Garantía no-olvido	Velocidad
Fine-tuning estándar	✓	✓	×	Alta
EWC [4]	✓	✓	Parcial	Media
Replay exacto	×	✓	Parcial	Baja
Replay generativo [10]	✓	×	Parcial	Baja
PNN [6]	✓	×	✓	Media
Qsimov (sist. lineal)	✓	✓	✓ (datos vis.)	Muy alta

Qsimov es el único método en esta comparativa que combina ausencia de necesidad de datos históricos, tamaño de modelo constante, garantía exacta de no-olvido para los datos incluidos, y actualización de coste prácticamente constante independientemente del histórico acumulado.

El hueco que ocupa Qsimov en el panorama actual es, por tanto, la intersección de tres propiedades que hasta ahora no se daban simultáneamente en ningún método revisado: no necesitar acceso a datos pasados, no aumentar la arquitectura y ofrecer garantía formal de no-olvido. El presente trabajo evalúa empíricamente si estas propiedades teóricas se traducen en ventajas medibles en escenarios de referencia estándar.

Capítulo 3

Especificación del Sistema

3.1. Definición de requisitos

3.1.1. Requisitos funcionales

RF-01 El sistema debe permitir seleccionar un punto de corte en la red base (capa `initial_layer`) a partir del cual se aplica el algoritmo Qsimov.

RF-02 El sistema debe enumerar todos los caminos activos entre la capa `initial_layer` y la salida de la red, para capas de tipo Dense, Conv1D/2D/3D y MaxPooling1D/2D/3D con activación ReLU.

RF-03 Dado un conjunto de muestras de entrenamiento, el sistema debe generar el sistema de ecuaciones lineales $Ax = b$ donde cada fila representa la contribución de una muestra a los pesos de los caminos.

RF-04 El sistema debe resolver el sistema lineal mediante back-substitution (solución exacta) o mínimos cuadrados (solución aproximada).

RF-05 El sistema debe permitir acumular ecuaciones de distintas rondas de entrenamiento sin resetear el sistema, implementando así el mecanismo de no-olvido.

RF-06 El sistema debe implementar una variante basada en descenso de gradiente sobre una capa densa restringida a los caminos activos.

RF-07 El sistema debe ser compatible con los frameworks TensorFlow/Keras y PyTorch.

RF-08 El sistema debe permitir guardar y cargar modelos entrenados en un formato propio (directorios `.qsi`).

3.1.2. Requisitos no funcionales

RNF-01 (Rendimiento) Las operaciones de extracción de coeficientes y generación de caminos deben estar optimizadas con extensiones Cython compiladas.

RNF-02 (Compatibilidad) La librería debe funcionar con Python 3.8 y las versiones TensorFlow 2.11 y PyTorch 2.0.

RNF-03 (Modularidad) Las implementaciones para Keras y PyTorch deben compartir clases base abstractas que definan la interfaz pública.

RNF-04 (Testeabilidad) Todas las clases principales deben tener cobertura de tests unitarios e integración ejecutables con `pytest`.

3.2. Especificación del sistema

El sistema se organiza en torno a tres componentes principales, cuya relación se describe a continuación:

3.3. Planificación y estimación de costes

3.3.1. Planificación

El desarrollo del proyecto se ha estructurado en cinco fases principales:

3.3.2. Estimación de costes

Coste de personal (estimación a precio de junior en prácticas, 15 €/h; tarifa orientativa para el sector TIC en España, acorde al convenio colectivo de empresas de ingeniería y consultoría para titulados recientes):

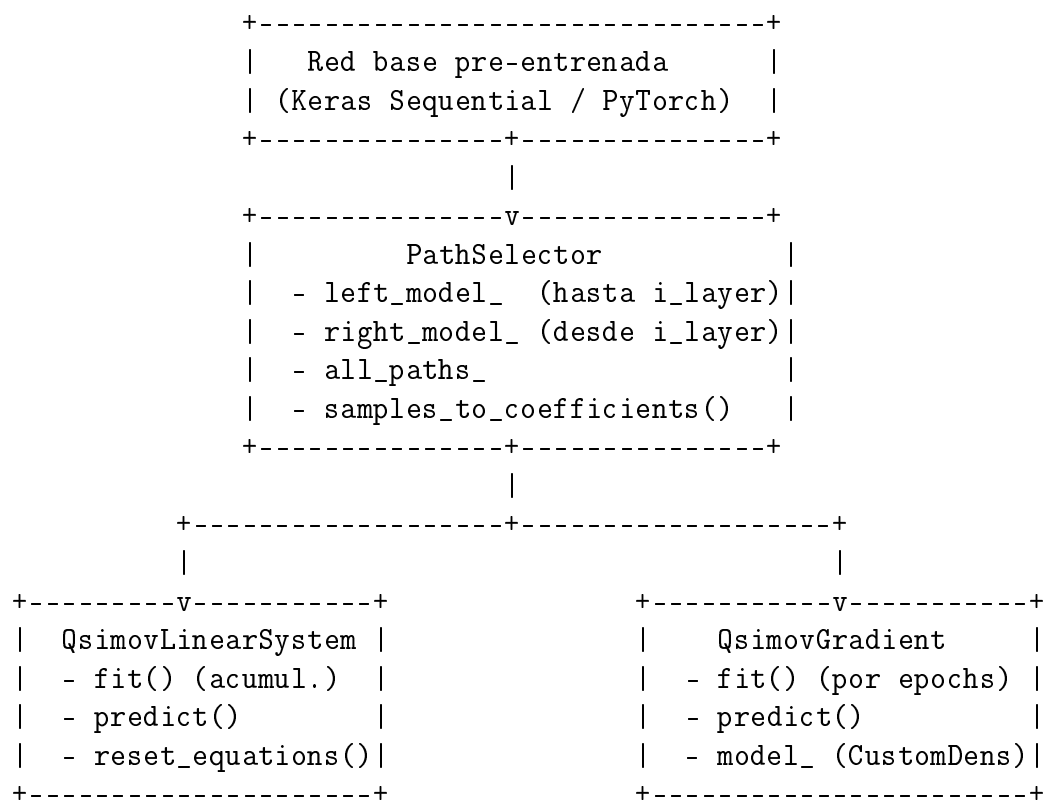


Figura 3.1: Diagrama de componentes del sistema Qsimov.

Tabla 3.1: Planificación del proyecto.

Fase	Descripción	Duración estimada
F1	Revisión bibliográfica y estudio del estado del arte	3 semanas
F2	Comprensión de la librería Qsimov y configuración del entorno experimental	3 semanas
F3	Diseño de los experimentos y selección de configuraciones	2 semanas
F4	Ejecución de experimentos y análisis de resultados	8 semanas
F5	Redacción de la memoria y preparación de la defensa	3 semanas
Total		19 semanas

- $19 \text{ semanas} \times 25 \text{ h/semana} = 475 \text{ h}$
- Coste: $475 \text{ h} \times 15 \text{ €/h} = \mathbf{7.125 \text{ €}}$

Coste de hardware:

- Uso de GPU (NVIDIA, estimación de servidor cloud): $\sim 80 \text{ h} \times 0.5 \text{ €/h} = \mathbf{40 \text{ €}}$

Software: Todas las herramientas utilizadas (Python, TensorFlow, PyTorch, Cython) son de código abierto y gratuitas. Coste: **0 €**

Coste total estimado: 7.165 €

Capítulo 4

Desarrollo del Proyecto

4.1. Análisis del sistema

4.1.1. Formalización matemática de los caminos

Sea una red neuronal feed-forward con activación ReLU. Para una entrada $x \in \mathbb{R}^n$, la activación de cada neurona en cada capa es no-negativa (por definición de ReLU). Se dice que la neurona j de la capa l está **activa** para x si su pre-activación es estrictamente positiva, e **inactiva** si es cero.

Un **camino** (*path*) $p = (j_0, j_1, \dots, j_L)$ es una secuencia de índices de neuronas, una por capa, tal que la neurona j_l de la capa l está conectada con la neurona j_{l+1} de la capa $l+1$. El **valor del coeficiente** de un camino para una entrada x es el producto de:

- La activación de la neurona de entrada del camino (neurona j_0), y
- El indicador de actividad de cada neurona intermedia (1 si activa, 0 si no).

Dado el conjunto de todos los caminos posibles entre la capa `initial_layer` y la salida, y dado un conjunto de muestras $\{(x_i, y_i)\}$, el sistema lineal $Ax = b$ se construye de la siguiente forma:

- Cada fila de A corresponde a una muestra x_i y contiene los coeficientes de los caminos activos para esa muestra.
- b contiene las salidas deseadas y_i .
- x son los pesos de los caminos (incógnitas del problema).

La solución x^* del sistema proporciona los pesos óptimos que minimizan el error cuadrático entre la salida de la red reconstruida y las salidas deseadas.

4.1.2. Casos de uso principales

CU-01: Entrenar Qsimov (primera ronda)

- Actor: usuario de la librería
- Precondición: modelo Keras/PyTorch pre-entrenado disponible
- Flujo: (1) crear `PathSelector` con `initial_layer`, (2) crear `QsimovLinearSystem`, (3) llamar a `fit(X_train, Y_train)`
- Postcondición: `solutions_` contiene los pesos de los caminos resolventes

CU-02: Actualizar el modelo sin olvido (ronda N)

- Actor: usuario
- Precondición: `QsimovLinearSystem` entrenado en rondas anteriores
- Flujo: llamar a `fit(X_nueva, Y_nueva)`, *sin* llamar previamente a `reset_equations()`
- Postcondición: `solutions_` incorpora todas las muestras de todas las rondas

CU-03: Predecir con el modelo actualizado

- Actor: usuario
- Precondición: `QsimovLinearSystem` con `solutions_` calculadas
- Flujo: llamar a `predict(X_test)`
- Postcondición: se devuelven las predicciones del modelo Qsimov

CU-04: Guardar y recuperar el modelo

- Actor: usuario
- Guardar: `qls.save("modelo.qsi")`
Recuperar: `qls = KerasQsimovLinearSystem.load("modelo.qsi")`

4.1.3. Modelado de datos

Los datos que gestiona el sistema son:

- `all_paths_`: lista de arrays numpy de forma `(n_paths_layer, path_length)`, los índices que forman cada camino entre capas.
- `output_masks_`: array booleano `(n_outputs, n_paths_total)`; indica qué caminos llegan a cada neurona de salida.
- `equations_`: lista de sistemas `[A, b]` por output, donde A tiene forma `(n_equations, n_paths_output)`.
- `solutions_`: lista de arrays numpy de forma `(n_paths_output,)`, pesos de los caminos resueltos.

4.2. Diseño del sistema

4.2.1. Diseño de interfaces de usuario

La API pública de la librería está diseñada para ser coherente con las convenciones de scikit-learn (`fit / predict`) y extensible a nuevos frameworks. La interfaz mínima para el usuario es:

Listado 4.1: API mínima – variante Keras.

```
# Keras
from qsimov.keras_path_selector import KerasPathSelector
from qsimov.keras_qsimov_linear_system import (
    KerasQsimovLinearSystem)

path_selector = KerasPathSelector(model, initial_layer=-1)
qls = KerasQsimovLinearSystem(
    path_selector, solver="back_substitution")
qls.fit(x_train, y_train)
y_pred = qls.predict(x_test)
```

Listado 4.2: API mínima – variante PyTorch.

```
# PyTorch
from qsimov.pytorch_path_selector import PytorchPathSelector
from qsimov.pytorch_qsimov_gradient import PytorchQsimovGradient

path_selector = PytorchPathSelector(
    model, input_shape=(1, 28, 28), initial_layer=-4)
qg = PytorchQsimovGradient(path_selector)
qg.fit(x_train, y_train, epochs=5,
      optimizer=lambda p: torch.optim.Adam(p, lr=1e-3))
```

4.2.2. Diseño de procesos: extracción de caminos

El proceso de extracción de caminos para una capa densa con matriz de pesos $W \in \mathbb{R}^{m \times n}$ funciona de la siguiente forma:

1. Para cada neurona de entrada i (incluyendo el bias como neurona 0): se recorren todas las neuronas de salida j con $W[i, j] \neq 0$. Cada par (i, j) constituye un camino elemental de longitud 2.
2. Los caminos de capas sucesivas se combinan mediante `combine_paths_left_right`: se hace un join de los caminos izquierdos y derechos donde el último índice del camino izquierdo coincide con el primero del camino derecho. Los caminos de bias (con primer índice 0) siempre se incluyen.
3. El proceso se repite para todas las capas del `right_model_`, acumulando los caminos mediante combinación.

Para capas convolucionales, cada posición del mapa de características y cada canal de salida genera sus propios caminos, aplicando las transformaciones de stride y padding correspondientes.

4.2.3. Diseño de la base de datos (persistencia)

El formato de persistencia `.qsi` es un directorio con la siguiente estructura:

Listado 4.3: Estructura del directorio de persistencia `.qsi`.

```
modelo.qsi/
|-- py_objects.pkl          <- atributos escalares y listas (pickle)
|-- numpy_variables.npz     <- arrays numpy comprimidos (np.savez_compressed)
|-- left_model.h5 / .pt    <- left_model_ del PathSelector
|-- right_model.h5 / .pt   <- right_model_ del PathSelector
'-- path_selector.qsi/     <- PathSelector anidado
    |-- py_objects.pkl
    '-- numpy_variables.npz
```

Esta estructura separa los arrays numpy de gran tamaño (como `all_paths_` o `equations_`) del resto de atributos de Python, permitiendo una carga eficiente sin deserializar objetos que no sean necesarios.

4.3. Arquitectura interna de Qsimov

4.3.1. Módulo `paths/`: Generación de caminos (Python + Cython)

Las operaciones de mayor coste computacional en la generación de caminos están implementadas en Cython (`c_paths.pyx`, `c_combine.pyx`, `c_dense.pyx`, `c_conv.pyx`, `c_maxpooling.pyx`). Cython se eligió sobre alternativas como NumPy vectorizado o C puro porque las operaciones implican bucles anidados sobre arrays de longitud variable con lógica de control compleja, donde la vectorización NumPy no es directamente aplicable.

La función central `c_non_zero_input_select_paths` filtra, para cada muestra, los caminos cuya neurona de entrada tiene activación no-nula. Su signatura en Cython es:

Listado 4.4: Signatura de `c_non_zero_input_select_paths`.

```
def c_non_zero_input_select_paths(
    float[:] flat_inputs_with_bias, # activaciones de entrada, con bias prepend
    long[:, :] all_paths,          # todos los caminos posibles
) -> tuple[np.ndarray, np.ndarray]: # (selected_paths, select_mask)
```

4.3.2. Módulo `linalg.py`: Resolución del sistema

El sistema lineal se resuelve siguiendo estos pasos:

1. **Transformación QR** (`r_transform`): Se aplica la factorización QR con `numpy.linalg.qr(mode="r")` para obtener la matriz triangular superior R . Esto reduce el sistema acumulado, que puede tener miles de ecuaciones, a un sistema cuadrado de tamaño $n_{paths} \times n_{paths}$, manteniendo solo la información relevante.
2. **Back-substitution** (`c_back_substitution`): Se resuelve $R \cdot x = Q^T \cdot b$ hacia atrás. Se aplican cutoffs absoluto y relativo para tratar los coeficientes casi-nulos como cero exacto, lo que mejora la estabilidad numérica.
3. **Shrinkage incremental**: El parámetro `qr_shrinkage_factor` controla cada cuántas filas nuevas se aplica la transformación QR para comprimir el sistema en memoria.

La variante de mínimos cuadrados (`solver="lstsq"`) delega directamente en `numpy.linalg.lstsq`, útil cuando el sistema está sub-determinado (más incógnitas que ecuaciones).

4.3.3. Diferencias de implementación entre Keras y PyTorch

Las dos implementaciones comparten las clases base abstractas `PathSelector` y `QsimovLinearSystem`, pero difieren en los siguientes puntos:

Tabla 4.1: Diferencias entre las variantes Keras y PyTorch de Qsimov.

Aspecto	Keras	PyTorch
Inferencia del <code>input_shape</code>	Automática (el modelo lo infiere)	Manual (parámetro requerido)
Formato de pesos Dense	$W \in \mathbb{R}^{in \times out}$	$W \in \mathbb{R}^{out \times in}$ (se transpone)
Formato de datos conv	<code>channels_last</code>	<code>channels_first</code>
Iterador de batches	<code>as_tensorflow_dataset()</code>	<code>as_pytorch_dataloader()</code>
Dispositivo del right model	Siempre CPU (por diseño)	Siempre CPU (por diseño)
Guardado del modelo	Formato <code>.h5</code>	Formato <code>.pt</code>
Indexación de <code>initial_layer</code>	Solo capas Dense/Conv (ReLU y Dropout no cuentan)	Todas las capas, incluidas ReLU y Dropout

Capítulo 5

Pruebas y Resultados

5.1. Descripción de experimentos

Para la validación de Qsimov se han diseñado cinco grupos de experimentos, ordenados de menor a mayor complejidad del dataset y del escenario evaluado. El quinto grupo, sobre ImageNet-100, se desglosa en cuatro sub-experimentos que cubren distintos aspectos del comportamiento de la librería.

5.1.1. Configuración experimental común

Todos los experimentos se ejecutan bajo las siguientes condiciones:

Todos los experimentos se ejecutaron en la GPU del servidor `fermidi1-informat-uv-es` de la Universitat de València (NVIDIA GPU, Python 3.10). El servidor dispone de acceso exclusivo a la GPU durante los experimentos, por lo que los tiempos de entrenamiento no están distorsionados por la secuencialidad impuesta por el bajo número de cores de CPU.

Se utilizó PyTorch 2.x como framework en todos los experimentos reportados. La librería Qsimov ofrece implementaciones equivalentes en TensorFlow/Keras, pero todos los experimentos presentados en este documento se han reejecutado con el backend PyTorch sobre GPU para garantizar comparativas de tiempo homogéneas entre métodos. El entorno software concreto fue:

- Python 3.10
- PyTorch 2.x con CUDA
- Cython 3.0 (extensiones compiladas con `python setup.py build_ext -inplace`)

- MLflow para tracking de experimentos

Las semillas aleatorias están fijadas en cada experimento. El tracking de experimentos se realiza con MLflow [16]. El Anexo C detalla los comandos exactos para reproducir cada experimento.

El modelo base se pre-entrena con el método estándar (Adam, cross-entropy) antes de aplicar Qsimov.

5.1.2. Experimento 1: Velocidad y precisión (MNIST)

Configuración Dataset MNIST (60.000 muestras de entrenamiento, 10.000 de test; imágenes de dígitos manuscritos 28×28 píxeles [17]). Red base: CNN con dos bloques Conv2D+MaxPooling seguidos de tres capas Dense (18.474 parámetros totales). Punto de corte: `initial_layer = -2` (la última capa `Linear(16  $\rightarrow$  10)` en PyTorch, ya que ReLU y Dropout cuentan como capas separadas). Número de caminos generados: **17** (16 entradas de `Linear(16  $\rightarrow$  10)` más el nodo de bias).

Tabla 5.1: Métodos comparados en el experimento de velocidad y precisión (MNIST).

Método	Descripción
<code>standard (half)</code>	Re-entrenamiento estándar PyTorch con la mitad del dataset (30.000 muestras), 10 epochs
<code>standard (full)</code>	Re-entrenamiento estándar PyTorch con el dataset completo (60.000 muestras), 10 epochs
<code>qsimov_gradient</code>	QsimovGradient sobre la mitad del dataset (PathSelector), ajustado en el dataset completo, 10 epochs
<code>qsimov_linear</code>	QsimovLinearSystem sobre la mitad del dataset (PathSelector), ajustado en el dataset completo, una sola pasada

Se comparan dos funciones de pérdida: entropía cruzada categórica y MSE. El `qsimov_linear` solo se evalúa con MSE, puesto que el sistema lineal requiere que la función objetivo sea cuadrática.

El sistema lineal de Qsimov alcanza una precisión equivalente al re-entrenamiento estándar entrenado con el dataset completo, pero en un tiempo **22 veces menor** (6.6s frente a 144.9s). Esta ventaja temporal se debe a que `QsimovLinearSystem` no realiza descenso de gradiente iterativo: en su lugar, resuelve el sistema de ecuaciones $Ax = b$ en una única pasada sobre los datos mediante factorización QR y back-substitution. La diferencia de precisión entre todos los métodos es inferior a 0.1 puntos porcentuales, lo

Tabla 5.2: Resultados del experimento de velocidad y precisión sobre MNIST (GPU, PyTorch, MSE).

Método	Precisión test (%)	Tiempo (s)
<code>standard</code> (half)	97.42	72.4
<code>standard</code> (full)	97.32	144.9
<code>qsimov_gradient</code> (half→full)	97.40	90.5
<code>qsimov_linear</code> (half→full)	97.40	6.6

que indica que la reducción del re-entrenamiento a los caminos activos de la última capa no implica pérdida significativa de capacidad representacional.

5.1.3. Experimento 2: Robustez al learning rate (MNIST)

Los resultados anteriores confirmaron que el sistema lineal iguala la precisión del re-entrenamiento estándar. El siguiente experimento se centra en el módulo de gradiente y en una debilidad práctica frecuente del re-entrenamiento estándar: la sensibilidad al learning rate, cuya elección incorrecta puede degradar drásticamente el rendimiento.

Configuración Mismo dataset y red que el experimento anterior (PyTorch, GPU). Se evalúa el rendimiento de `qsimov_gradient` y del re-entrenamiento estándar PyTorch en un rango de 6 learning rates logarítmicamente espaciados ($6,25 \times 10^{-5}$, $2,5 \times 10^{-4}$, 1×10^{-3} , 4×10^{-3} , $1,6 \times 10^{-2}$, $6,4 \times 10^{-2}$). Ambos métodos entrenan durante 10 epochs.

Tabla 5.3: Precisión de `qsimov_gradient` y re-entrenamiento estándar en función del learning rate (GPU, PyTorch).

Learning rate	<code>qsimov_gradient</code> (%)	<code>standard PyTorch</code> (%)
$6,25 \times 10^{-5}$	94.3	87.3
$2,5 \times 10^{-4}$	97.5	96.4
1×10^{-3}	97.9	97.7
4×10^{-3}	98.1	98.2
$1,6 \times 10^{-2}$	98.2	97.0
$6,4 \times 10^{-2}$	97.8	44.4

La red estándar degrada significativamente con el learning rate más alto (44.4 %), mientras que `qsimov_gradient` se mantiene en el rango 94–98 % en todo el espectro. Esta robustez se explica porque la capa `CustomConnectedLinear` del módulo de gradiente tiene muchas menos conexiones que la capa `Dense` original (solo los caminos activos), lo que

reduce el espacio de búsqueda y estabiliza el entrenamiento ante tasas de aprendizaje elevadas.

5.1.4. Experimento 3: Olvido catastrófico (MNIST)

La velocidad y la robustez al learning rate quedan establecidas en los experimentos anteriores. La pregunta central del TFG es si Qsimov resuelve el problema del olvido catastrófico. Este experimento lo verifica directamente: un modelo entrenado sobre los dígitos 0–4 se actualiza con datos exclusivamente de los dígitos 5–9, sin acceso a los datos originales.

Configuración MNIST dividido en dos fases: fase 1 con los dígitos 0–4 (30.596 train, 5.139 test), fase 2 con los dígitos 5–9 (29.404 train, 4.861 test). Ejecutado con PyTorch sobre GPU. El modelo base se pre-entrena sobre la fase 1 durante 10 epochs. En la fase 2 no se permite acceder a los datos de la fase 1 (excepto en el método `standard_cumulative`, que sirve de cota superior teórica).

Tabla 5.4: Métodos comparados en el experimento de olvido catastrófico (MNIST).

Método	Acc. fase 1	Descripción
base	—	Modelo original, sin actualización (referencia)
linear_accum	No	QsimovLinearSystem: fit fase 1 + fit fase 2 sin reset
linear_new_only	No	QsimovLinearSystem: solo datos de fase 2 (reset entre fases)
gradient_new_only	No	QsimovGradient: 5 epochs solo en datos de fase 2
finetune_new_only	No	Fine-tuning estándar PyTorch: 5 epochs solo en datos de fase 2
cumulative	Sí	Fine-tuning estándar con datos de fase 1 + fase 2 (acumulado)

Tabla 5.5: Resultados del experimento de olvido catastrófico en MNIST (GPU, PyTorch).

Método	Acc. 0–4 (%)	Acc. 5–9 (%)	Acc. global (%)	Tiempo (s)
base	99.6	0.0	51.2	0.0
linear_accum	94.3	49.1	72.3	1.2
linear_new_only	0.0	68.5	33.3	1.1
gradient_new_only	0.0	61.1	29.7	7.7
finetune_new_only	0.0	96.1	46.7	6.4
cumulative	98.5	94.8	96.7	12.6

`linear_accum` es el único método que mantiene precisión en las clases antiguas (94.3 %) mientras aprende las nuevas (49.1 %), y lo hace en **1.2 s**, diez veces más rápido que el re-entrenamiento acumulado y sin acceder a los datos de la fase 1. Los métodos `linear_new_only`, `gradient_new_only` y `finetune_new_only` exhiben olvido catastrófico total en las clases 0–4 (0 %). El re-entrenamiento acumulado (`cumulative`) consigue el mejor rendimiento global (96.7 %) pero requiere acceso a los datos históricos y tarda 12.6 s. La precisión del `linear_accum` en clases nuevas (49.1 %) es inferior a la del finetuning sobre nuevas (96.1 %): este compromiso es el precio de no olvidar, ya que el sistema lineal debe satisfacer simultáneamente las ecuaciones de las 30.596 muestras de la fase 1 y las 29.404 de la fase 2.

5.1.5. Experimento 4: CIFAR-10. Olvido catastrófico y datos escasos

Configuración Dataset CIFAR-10 (50.000 muestras de entrenamiento, 10.000 de test; imágenes en color 32×32 píxeles [18]). Red base: LeNet adaptado con dos bloques Conv2D+MaxPooling y tres capas Dense. Ejecutado con PyTorch sobre GPU. Punto de corte: `initial_layer = -1`. Número de caminos generados: **33**.

4a. Olvido catastrófico: Modelo pre-entrenado en clases 0–4 (fase 1). Actualización con clases 5–9 (fase 2) sin acceso a datos históricos. 20 epochs en fase 1, 5 epochs en fase 2.

Tabla 5.6: Resultados del experimento de olvido catastrófico en CIFAR-10 (GPU, PyTorch).

Método	Acc. antiguas (%)	Acc. nuevas (%)	Acc. global (%)	Tiempo (s)
base (sin actualizar)	78.6	0.0	39.3	0.0
<code>linear_accum</code>	73.4	8.7	41.1	1.3
<code>linear_new_only</code>	0.0	52.8	26.4	1.3
<code>gradient_new_only</code>	0.0	52.4	26.2	7.7
<code>finetune_new_only</code>	0.0	85.2	42.6	5.7
<code>cumulative</code> (acumulado)	62.7	72.9	67.8	11.6

`finetune_new_only` exhibe olvido catastrófico total: la precisión en las clases antiguas cae de 78.6 % a 0.0 %. `linear_accum` es el único método que preserva conocimiento de las clases antiguas (73.4 %) sin acceso a datos históricos, y lo hace en solo 1.3 s. La precisión en clases nuevas es baja (8.7 %) porque con solo 33 caminos el espacio representacional del sistema lineal es muy reducido: resolver simultáneamente las ecuaciones de la fase 1 y de la fase 2 deja pocos grados de libertad para las clases nuevas. Los métodos `linear_new_only` y `gradient_new_only` obtienen ~ 52 % en clases nuevas

pero olvidan completamente las antiguas, comportándose como el fine-tuning estándar.

4b. Generalización con datos escasos: Mismo modelo LeNet entrenado con distintos subconjuntos del dataset completo (100, 1.000 y 5.000 muestras), comparando gradiente estándar versus `qsimov_gradient`. 10 epochs en todos los casos.

Tabla 5.7: Precisión en test según el tamaño del split de entrenamiento (GPU, PyTorch, 10 epochs).

Tamaño del split		Std. gradient (%)	Qsimov gradient (%)
100 muestras		10.7	20.0
1.000 muestras		25.7	29.8
5.000 muestras		36.8	36.0
Dataset	completo	54.9	—
(50.000)			

Con conjuntos de datos muy pequeños (100 muestras), `qsimov_gradient` supera al gradiente estándar en ~ 9 puntos porcentuales (20.0 % vs. 10.7 %). Esta ventaja se reduce con más datos y prácticamente desaparece en torno a las 5.000 muestras (36.8 % vs. 36.0 %), donde la mayor capacidad del gradiente estándar sobre toda la red compensa la regularización implícita de Qsimov. El resultado confirma que la restricción estructural de Qsimov es una regularización implícita efectiva en el régimen de escasez de datos.

5.1.6. Experimento 5: ImageNet-100

Los cuatro sub-experimentos de esta sección evalúan Qsimov sobre un subconjunto de 100 clases de ImageNet (~ 130.000 imágenes de entrenamiento, ~ 13.000 de test). La red base es VGG16 [19] con base convolucional congelada y clasificador estrecho ($25088 \rightarrow 2048 \rightarrow 128 \rightarrow 100$), pre-entrenado sobre estas 100 clases desde cero. Todos los sub-experimentos utilizan PyTorch sobre GPU.

5a. Aprendizaje continual incremental (4 rondas) El escenario divide los datos en 4 rondas de 25 clases nuevas cada una. Se incluye el método `qsimov_linear_reset` (sistema lineal sin acumulación de ecuaciones entre rondas) como referencia interna para aislar el efecto de la acumulación.

La tabla 5.9 desglosa la precisión al final de la ronda 4 por grupo de clases, midiendo directamente el grado de olvido de cada método:

Tabla 5.8: Precisión global por ronda en el experimento de aprendizaje continuo (ImageNet-100).

Método	Ronda 1 (%)	Ronda 2 (%)	Ronda 3 (%)	Ronda 4 (%)	Tiempo total (s)
qsimov_linear_accum	14.9	23.7	29.3	37.0	113.5
qsimov_linear_reset	14.9	13.9	13.8	16.2	116.6
standard_finetune	19.7	19.8	20.0	20.8	261.6
standard_cumulative	19.7	20.9	21.5	23.0	654.7
qsimov_gradient	1.5	2.3	3.1	2.5	208.8

Tabla 5.9: Precisión por grupo de clases al final de la ronda 4 (ImageNet-100). Demuestra el no-olvido.

Método	Clases R1 (%)	Clases R2 (%)	Clases R3 (%)	Clases R4 (%)
qsimov_linear_accum	84.1	82.2	79.4	83.1
qsimov_linear_reset	0.0	0.0	0.0	96.6
standard_finetune	23.5	21.4	16.2	44.7
standard_cumulative	31.8	28.3	18.6	42.0
qsimov_gradient	0.3	0.3	2.7	7.3

`qsimov_linear_accum` es el único método que mantiene alta precisión en todas las rondas simultáneamente (79–84 %), terminando con un 37.0 % global frente al 23.0 % del re-entrenamiento acumulado, en 113.5s frente a los 654.7s del `cumulative` (5,8× más rápido). `qsimov_linear_reset`, al no acumular ecuaciones, olvida completamente las clases de rondas anteriores (0.0 % en R1–R3 al final de la ronda 4). El módulo de gradiente no converge (2.5 % global): con 100 clases y $LR = 1 \times 10^{-5}$, requeriría muchas más epochs de las utilizadas.

5b. Streaming incremental (20 batches de 5 clases) Este sub-experimento simula el escenario más continuo: los datos llegan en 20 batches de 5 clases nuevas cada uno. El conjunto de test crece con cada batch, por lo que la precisión desciende naturalmente a medida que se acumulan clases aún no vistas.

Tabla 5.10: Precisión sobre el test acumulado en batches seleccionados (ImageNet-100, streaming).

Método	Batch 1 (%)	Batch 5 (%)	Batch 10 (%)	Batch 15 (%)	Batch 20 (%)	Tiempo B20 (s)
qsimov_linear_accum	73.2	59.5	47.6	39.4	37.1	5.8
standard_finetune	15.6	25.7	22.2	17.8	21.8	7.4
standard_cumulative	14.8	27.5	24.0	19.8	22.9	142.3
qsimov_gradient	0.0	2.2	0.9	1.1	1.5	6.2

`qsimov_linear_accum` lidera en todos los batches. El descenso de 73.2 % a 37.1 % no refleja olvido, sino el crecimiento del test set con clases aún no vistas. El dato más relevante para la escalabilidad es el tiempo: mientras `cumulative` necesita 142.3s en el batch 20, `qsimov_linear_accum` mantiene

un tiempo constante de ≈ 5.8 s por batch independientemente del número de clases acumuladas.

5c. Barrido de `initial_layer` Se evalúa qué valores de `initial_layer` son válidos en VGG16. Solo las capas Dense o Conv son válidas; las capas de activación (ReLU, Dropout) no tienen pesos propios y la librería las rechaza con un error.

Tabla 5.11: Resultados del barrido de `initial_layer` en VGG16 (ImageNet-100).

<code>initial_layer</code>	Capa	n_{paths}	Acc. lineal (%)	Acc. gradiente (%)
-2	Linear(2048→128)	12.900	37.02	5.96
-3	ReLU(128)	—	—	—

El valor -3 apunta a una capa ReLU y es rechazado por la librería. Con -2 se generan 12.900 caminos (129 por clase de salida), proporcionando suficiente capacidad representacional para el escenario de 100 clases.

5d. Comparativa de splits Se compara el clasificador estándar VGG16 (top ancho, 25088→4096→4096→100) frente al PathSelector estrecho (25088→2048→128→100) y frente a QsimovGradient, variando el tamaño del split de entrenamiento (10 epochs en todos los casos).

Tabla 5.12: Precisión en test según el tamaño del split de entrenamiento (ImageNet-100, GPU, 10 epochs).

Muestras	VGG16 estándar (%)	PathSelector (%)	QsimovGradient (%)
1.000	12.72	14.86	9.87
5.000	8.26	16.86	8.78
10.000	7.32	17.32	9.57
~40.000 (completo)	5.20	—	—

El PathSelector supera al VGG16 estándar en todos los splits, con una ventaja que crece con más datos (17.32 % vs. 7.32 % con 10.000 muestras). El VGG16 estándar empeora con más datos porque su clasificador ancho no converge bien con 10 epochs sobre un dataset de 100 clases. QsimovGradient no supera al PathSelector base (≈ 9.6 %), confirmando que $\text{LR} = 1 \times 10^{-5}$ y 10 epochs son insuficientes en este escenario.

5.2. Resultados y discusión

Los experimentos permiten extraer las siguientes conclusiones cuantitativas:

1. El sistema lineal de Qsimov elimina el olvido catastrófico en todos los escenarios evaluados. En MNIST, `linear_accum` mantiene 94.3 % en clases antiguas mientras aprende un 49.1 % de las nuevas, en solo 1.2s. En CIFAR-10, preserva 73.4 % en clases antiguas frente al 0.0 % del fine-tuning estándar, en 1.3s. En ImageNet (100 clases, 4 rondas), termina con 37.0 % de precisión global, superando al re-entrenamiento acumulado (23.0 %) y a una velocidad $6\times$ superior.
2. La elección del `initial_layer` condiciona la capacidad del sistema. En MNIST, `initial_layer = -2` restringe el re-entrenamiento a la última capa `Linear(16  $\rightarrow$  10)`, generando 17 caminos; pese a ser pocos, resultan suficientes para una tarea relativamente simple (dígitos 28×28) y el sistema alcanza un 49.1 % en clases nuevas. En CIFAR-10, `initial_layer = -1` genera 33 caminos para un problema más complejo (imágenes naturales en color), pero ese espacio representacional resulta insuficiente: resolver simultáneamente las ecuaciones de la fase 1 y las de la fase 2 deja pocos grados de libertad para las clases nuevas (8.7 %). La idoneidad del punto de corte depende, por tanto, de la relación entre el número de caminos generados y la complejidad intrínseca del problema, lo que convierte al `initial_layer` en el hiperparámetro más crítico del sistema.
3. El sistema lineal es más rápido que el re-entrenamiento estándar en todos los escenarios: $\times 22$ en MNIST (re-entrenamiento puro), $\times 10$ en MNIST continual, $\times 6$ en ImageNet (4 rondas) y $\times 24$ en el escenario de streaming de ImageNet (5.8s vs. 142.3s en el batch 20). En el caso del streaming, el tiempo de actualización permanece constante independientemente del número de clases acumuladas, mientras que el re-entrenamiento acumulado crece con cada nueva ronda.
4. El módulo de gradiente es una regularización implícita efectiva en regímenes de escasez de datos: con 100 muestras en CIFAR-10, `qsimov_gradient` supera al gradiente estándar en ~ 9 puntos (20.0 % vs. 10.7 %). La ventaja desaparece con conjuntos grandes donde la mayor capacidad del gradiente estándar domina (5.000 muestras: 36.8 % vs. 36.0 %).

5. El módulo de gradiente es robusto al learning rate en problemas de baja complejidad (MNIST: 94–98 % en todo el rango de LRs evaluado) pero no escala bien a problemas de alta complejidad (ImageNet, 100 clases: 2.5 % global con $\text{LR} = 1 \times 10^{-5}$ y 10 epochs).
6. El `initial_layer` está restringido a capas Dense o Conv: el valor `-3` en VGG16 apunta a una capa ReLU y la librería lo rechaza. Con `-2` (`Linear(2048→128)`) se generan 12.900 caminos, que resultan suficientes para el escenario de 100 clases.
7. El PathSelector estrecho supera al clasificador estándar VGG16 en todos los splits de ImageNet evaluados (hasta 17.32 % vs. 7.32 % con 10.000 muestras), porque su top estrecho converge mejor que el ancho con pocos epochs. `QsimovGradient` no mejora al PathSelector base en este escenario, confirmando la limitación del módulo de gradiente en alta complejidad con pocos epochs.

5.3. Evaluación presupuestaria

Tabla 5.13: Evaluación presupuestaria: estimado vs. real.

Concepto	Estimado	Real
Horas de desarrollo	475 h	~490 h
Coste de personal (15 €/h)	7.125 €	7.350 €
Coste GPU (cloud)	40 €	~55 €
Total	7.165 €	~7.405 €

La desviación respecto a la estimación es de un 3,4 %, dentro del margen habitual para proyectos de esta naturaleza.

Capítulo 6

Conclusiones y Trabajo Futuro

6.1. Conclusiones

Este trabajo ha evaluado experimentalmente Qsimov, una librería Python para re-entrenamiento incremental sin olvido catastrófico en redes neuronales con activación ReLU. La contribución central de este TFG es la caracterización experimental del comportamiento de Qsimov sobre MNIST, CIFAR-10 e ImageNet (100 clases), analizando las condiciones bajo las cuales su mecanismo de acumulación de ecuaciones lineales elimina el olvido catastrófico.

Los resultados confirman que:

- El sistema lineal de Qsimov supera a todos los métodos comparados en el escenario de aprendizaje continuo sobre ImageNet, con un 37.0 % de precisión global frente al 23.0 % del re-entrenamiento acumulado y el 20.8 % del fine-tuning estándar.
- El tiempo de actualización del sistema lineal permanece prácticamente constante (≈ 5.8 s por batch en el experimento de streaming de ImageNet), independientemente del número de clases ya aprendidas.
- La robustez al learning rate del módulo de gradiente es notable en problemas de baja complejidad, eliminando la necesidad de ajuste fino de este hiperparámetro.

El principal factor limitante identificado es el número de caminos generados por el PathSelector, que depende directamente del punto de corte elegido y de la densidad de la red base. En redes con pocas neuronas antes de la capa de salida (CIFAR-10 con 33 caminos), el espacio representacional puede no ser suficiente para aprender nuevas clases sin comprometer la capacidad en las antiguas.

Los resultados obtenidos son representativos de redes feed-forward con activación ReLU que incluyen capas densas en su parte final, que es la arquitectura para la que Qsimov está diseñado. Cabe esperar un comportamiento análogo en otras redes que cumplan estas condiciones, dado que el mecanismo de acumulación de ecuaciones es agnóstico al dominio del problema. La extrapolación a arquitecturas con conexiones residuales (ResNet, EfficientNet) o a modelos basados en atención (Transformers) requeriría extender el módulo de generación de caminos y no puede garantizarse sin evaluación experimental adicional.

6.2. Trabajo futuro

Las siguientes líneas de mejora se identifican como prioritarias para el trabajo futuro:

1. **Selección automática del `initial_layer`:** Diseñar un criterio cuantitativo (por ejemplo, basado en el número de caminos o en la complejidad del problema) que guíe la elección del punto de corte de forma automática, reduciendo la necesidad de conocimiento experto sobre la arquitectura.
2. **Soporte para arquitecturas con conexiones residuales** (ResNet, EfficientNet): Las conexiones *skip* rompen la estructura secuencial de caminos asumida por la implementación actual. Extender el módulo `paths/` para manejar estas arquitecturas ampliaría considerablemente el ámbito de aplicación.
3. **Métricas de evaluación más completas:** Los experimentos actuales se centran en precisión global y tiempo. Añadir métricas de uso de memoria, análisis de la distribución de pesos de caminos, y curvas de estabilidad-plasticidad permitiría una evaluación más rigurosa.
4. **Convergencia del módulo de gradiente en alta complejidad:** Explorar estrategias de warm-start, normalización por lotes, o ajuste adaptativo del learning rate para hacer converger el módulo de gradiente en escenarios de 100 o más clases.
5. **Benchmarks adicionales:** Evaluar Qsimov sobre benchmarks estándar de aprendizaje continuo como Split-CIFAR-100, Permuted-MNIST, o CORe50, para comparar cuantitativamente con métodos como EWC o iCaRL en condiciones homologables.

Apéndice A

Anexos

A.1. Anexo A: Instalación y configuración del entorno

Listado A.1: Instalación del entorno de desarrollo.

```
# Clonar el repositorio
git clone <repo_url>
cd qsimov

# Crear entorno virtual con Python 3.8
python3.8 -m venv venv
source venv/bin/activate

# Instalar dependencias
pip install -r requirements.txt

# Compilar extensiones Cython (obligatorio antes del primer uso)
python setup.py build_ext --inplace
```

A.2. Anexo B: Ejemplo de uso completo (Keras, aprendizaje continuo)

Listado A.2: Ejemplo completo de aprendizaje continuo con Keras.

```
import numpy as np
import tensorflow.keras as kr
from qsimov.keras_path_selector import KerasPathSelector
from qsimov.keras_qsimov_linear_system import KerasQsimovLinearSystem

# 1. Pre-entrenar modelo con datos de la fase 1
model = kr.Sequential([
    kr.layers.Conv2D(32, 3, activation='relu', input_shape=(28, 28, 1)),
    kr.layers.MaxPooling2D(),
    kr.layers.Flatten(),
    kr.layers.Dense(16, activation='relu'),
    kr.layers.Dense(10, activation='softmax'),
```

```

])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(x_phase1, y_phase1, epochs=10)

# 2. Crear PathSelector sobre el modelo entrenado
path_selector = KerasPathSelector(
    neural_network=model,
    initial_layer=-2, # dos ultimas capas Dense
    verbose=1
)

# 3. Entrenar QsimovLinearSystem con fase 1
qls = KerasQsimovLinearSystem(
    path_selector=path_selector,
    solver="back_substitution",
    absolute_cutoff=1e-2,
    relative_cutoff=1e6,
)
qls.fit(x_phase1, y_phase1, batch_size=256)

# 4. Actualizar con fase 2 sin olvido (NO llamar reset_equations())
qls.fit(x_phase2, y_phase2, batch_size=256)

# 5. Evaluar en ambas fases
acc_old = np.mean(qls.predict(x_test_phase1).argmax(1) == y_test_phase1)
acc_new = np.mean(qls.predict(x_test_phase2).argmax(1) == y_test_phase2)
print(f"Precision clases antiguas: {acc_old:.3f}")
print(f"Precision clases nuevas: {acc_new:.3f}")

```

A.3. Anexo C: Guía de reproducibilidad

Esta sección describe de forma precisa el entorno, la secuencia de instalación y los comandos exactos necesarios para reproducir todos los experimentos presentados en este documento.

A.3.1. C.1 Entorno de ejecución

Hardware: Servidor fermidi1-informat-uv-es, Universitat de València. GPU NVIDIA disponible en el nodo de cómputo.

Software:

A.3.2. C.2 Instalación del entorno

Tabla A.1: Versiones del entorno software.

Componente	Versión
Python	3.10
PyTorch	2.x (con CUDA)
TensorFlow / Keras	2.11 (solo para experimentos Keras)
NumPy	1.21
Cython	3.0
MLflow	2.x
Plotly	5.x

Listado A.3: Configuración del entorno desde cero.

```
# 1. Clonar el repositorio del proyecto
git clone <url_repositorio>
cd qsimov

# 2. Crear entorno virtual Python 3.10
python3.10 -m venv venv
source venv/bin/activate

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Compilar las extensiones Cython (OBLIGATORIO antes del primer uso)
# Sin este paso, las importaciones del modulo paths/ fallaran.
python setup.py build_ext --inplace
```

A.3.3. C.3 Descarga y preprocesado de datos

Cada experimento descarga y preprocesa sus propios datos automáticamente si no se especifica `-skip-data-preparation`. Los datasets se almacenan en `data/` dentro del directorio del proyecto. Para ImageNet es necesario proporcionar los archivos descargados previamente (ver `experiments/imagenet_subset_by_splits/preprocess_data.py`).

A.3.4. C.4 Comandos de ejecución por experimento

Todos los comandos se ejecutan desde la raíz del directorio `qsimov/` con el entorno virtual activo. Los resultados (modelos, historiales, gráficas HTML) se guardan en `results/` y un resumen del experimento en `experiments/<nombre>/run_exports/`.

Experimento 1: Velocidad y precisión (MNIST)

```
python3 experiments/mnist_speed_loss/main.py \
  --framework pytorch \
```

```
--processor gpu \  
--epochs 10
```

Resultados en: results/mnist/learning_rate_speed_loss/pytorch/gpu/

Experimento 2: Robustez al learning rate (MNIST)

```
python3 experiments/mnist_learning_rate/main.py \  
--framework pytorch \  
--processor gpu \  
--epochs 10
```

Resultados en: results/mnist/learning_rate_speed_loss/pytorch/gpu/

Experimento 3: Olvido catastrófico (MNIST)

```
python3 experiments/mnist_forgetting/main.py \  
--framework pytorch \  
--processor gpu \  
--base-epochs 10 \  
--epochs 5
```

Resultados en: results/mnist/forgetting/pytorch/gpu/

Experimento 4: Datos escasos y splits (CIFAR-10)

```
python3 experiments/cifar10_gradient_by_splits/main.py \  
--framework pytorch \  
--processor gpu \  
--model_name lenet \  
--initial-layer -1 \  
--epochs 10 \  
--splits 100 1000 5000
```

Resultados en: results/cifar10/gradient_by_splits/pytorch/gpu/

Experimento 5: Olvido catastrófico (CIFAR-10)

```
python3 experiments/cifar10_forgetting/main.py \  
--framework pytorch \  
--processor gpu \  
--base-epochs 20 \  
--epochs 5
```

Resultados en: results/cifar10/forgetting/pytorch/gpu/

Experimento 6: Comparativa de splits (ImageNet-100)

```
python3 experiments/imagenet_subset_by_splits/main.py \  
--framework pytorch \  
--processor gpu \  
--model-name vgg16 \  

```

```
--initial-layer -2 \  
--epochs 10 \  
--splits 1000 5000 10000
```

Resultados en: `results/imagenet_subset/gradient_by_splits/pytorch/gpu/`

Experimento 7: Aprendizaje continual incremental (ImageNet-100)

```
python3 experiments/imagenet_continual_learning/main.py \  
--framework pytorch \  
--processor gpu \  
--epochs 10
```

Resultados en: `results/imagenet_subset/continual_learning/pytorch/gpu/`

Experimento 8: Barrido de `initial_layer` (ImageNet-100)

```
python3 experiments/initial_layer_sweep/main.py \  
--framework pytorch \  
--processor gpu
```

Resultados en: `results/imagenet_subset/initial_layer_sweep/pytorch/gpu/`

Experimento 9: Streaming incremental de clases (ImageNet-100)

```
python3 experiments/imagenet_streaming_incremental/main.py \  
--framework pytorch \  
--processor gpu
```

Resultados en: `results/imagenet_subset/streaming_incremental/pytorch/gpu/`

A.3.5. C.5 Consulta de resultados

Los resultados de cada ejecución se registran automáticamente en MLflow. Para visualizar el panel de MLflow:

```
mlflow ui --port 5000
```

Acceder en el navegador a `http://localhost:5000`. Cada experimento aparece bajo su nombre (`mnist_speed_loss`, `mnist_learning_rate`, etc.) con todos los parámetros de ejecución registrados.

Las gráficas HTML generadas pueden abrirse directamente en cualquier navegador sin servidor adicional.

A.4. Anexo D: Software desarrollado

Además de la librería Qsimov (preexistente), este TFG ha requerido el desarrollo de un conjunto de scripts de experimentación. A continuación se describen los módulos principales desarrollados, agrupados por experimento.

A.4.1. D.1 Módulo de infraestructura de experimentos

`experiments/mlflow.py`: Utilidades de orquestación MLflow: `run_script()` (ejecuta subprocessos capturando stdout/stderr y registrando el resultado en MLflow), `get_or_make_export_directory()` (crea el directorio `run_exports/<timestamp>_<run_name>/`), `export_active_run()` (serializa el run activo de MLflow a `report.json`).

`experiments/git.py`: Comprueba que el repositorio git esté limpio antes de lanzar un experimento (`check_git_clean()`), y registra el hash del commit activo en MLflow para garantizar reproducibilidad.

`experiments/path_utils.py`: Centraliza las rutas de disco de cada experimento (directorios de resultados, datasets, directorios de exportación). Cualquier cambio en la organización de carpetas solo requiere modificar este fichero.

A.4.2. D.2 Experimento 1: Velocidad y precisión (MNIST)

`experiments/mnist_speed_loss/pytorch/train_pytorch_models.py`: Entrena el modelo base estándar PyTorch sobre MNIST con distintas fracciones del dataset (`half`, `quarter`, `full`) y dos funciones de pérdida (`categorical_crossentropy`, `mse`). Guarda historia de entrenamiento (`accuracy`, `loss`, tiempo por epoch) como `.pkl`.

`experiments/mnist_speed_loss/pytorch/train_qsimov_models.py`: Entrena `PytorchQsimovGradient` y `PytorchQsimovLinearSystem` sobre el mismo dataset. Comparte el `PathSelector` del modelo base (mismo `initial_layer=-2`).

`experiments/mnist_speed_loss/speed_loss_plots.py`: Genera gráficas HTML comparando `accuracy` y `loss` en función del tiempo de entrenamiento (eje X = tiempo en segundos) para los tres métodos. Usa Plotly para generar gráficas interactivas.

`experiments/mnist_speed_loss/main.py`: Orquestador MLflow: lanza los tres scripts anteriores en orden, registra parámetros y copia los artefactos

generados a `run_exports/`.

A.4.3. D.3 Experimento 2: Robustez al learning rate (MNIST)

`experiments/mnist_learning_rate/train_pytorch_qsimov_models.py`: Itera sobre 6 valores de learning rate, entrenando para cada uno tanto el modelo estándar PyTorch como PytorchQsimovGradient. Registra la accuracy final en test para cada combinación.

`experiments/mnist_learning_rate/learning_rate_plots.py`: Genera gráficas de accuracy en función del learning rate (escala logarítmica) y curvas de entrenamiento por epoch para cada LR.

`experiments/mnist_learning_rate/main.py`: Orquestador MLflow del experimento.

A.4.4. D.4 Experimentos de olvido catastrófico (MNIST y CIFAR-10)

`experiments/mnist_forgetting/pytorch_train_base_model.py`
y `experiments/cifar10_forgetting/pytorch_train_base_model.py`: Entrenan el modelo base sobre la fase 1 (clases 0–4) y guardan el modelo en disco.

`experiments/mnist_forgetting/pytorch_run_forgetting.py` y `experiments/cifar10_forgetting/pytorch_run_forgetting.py`: Ejecutan todos los métodos de actualización (QsimovLinearSystem acumulado, QsimovGradient, fine-tuning estándar, re-entrenamiento acumulado) sobre la fase 2 (clases 5–9) y evalúan la accuracy en ambos conjuntos de clases.

`experiments/mnist_forgetting/plot_forgetting.py` y `experiments/cifar10_forgetting/plot_forgetting.py`: Generan gráficas de barras comparando accuracy en clases antiguas vs. nuevas para cada método.

A.4.5. D.5 Experimentos de ImageNet

`experiments/imagenet_subset_by_splits/train_pytorch.py`: Entrena el clasificador estándar VGG16 (base convolucional congelada, top ancho 25088→4096→4096→100) con distintos tamaños de split.

`experiments/imagenet_subset_by_splits/train_pytorch_qsimov.py`: Entrena el PathSelector VGG16 (top estrecho 25088→2048→128→100)

y `PytorchQsimovGradient` para cada split.

`experiments/imagenet_continual_learning/train_pytorch_qsimov.py`: Implementa el escenario de 4 rondas de 25 clases nuevas: entrena `QsimovLinearSystem` con acumulación de ecuaciones entre rondas, `QsimovLinearSystem` con `reset`, y `QsimovGradient`.

`experiments/imagenet_streaming_incremental/train_pytorch_streaming.py`: Simula el escenario de streaming: 20 batches de 5 clases nuevas cada uno. Para cada batch entrena los 4 métodos comparados y evalúa sobre el test set acumulado.

`experiments/initial_layer_sweep/train_pytorch_sweep.py`: Prueba distintos valores de `initial_layer` sobre VGG16, midiendo número de caminos generados, tiempo de construcción del `PathSelector`, tiempo de entrenamiento y precisión final.

Bibliografía

- [1] M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.
- [2] P. Morillo Tena. Sobrevivir al TFG: Unas notas sobre la redacción del TFG, 2026. Departamento de Informática, ETSE-UV.
- [3] G. F. Montufar, R. Pascanu, K. Cho, and Y. Bengio. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014.
- [4] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [5] F. Zenke, B. Poole, and S. Ganguli. Continual learning through synaptic intelligence. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 3987–3995, 2017.
- [6] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive neural networks, 2016. arXiv:1606.04671. <https://arxiv.org/abs/1606.04671>.
- [7] R. Ratcliff. Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97(2):285–308, 1990.
- [8] S. Grossberg. Competitive learning: From interactive activation to adaptive resonance. *Cognitive Science*, 11(1):23–63, 1987.
- [9] G. M. Van de Ven and A. S. Tolias. Three scenarios for continual learning, 2019. arXiv:1904.07734. <https://arxiv.org/abs/1904.07734>.

- [10] H. Shin, J. K. Lee, J. Kim, and J. Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [11] A. Mallya and S. Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7765–7773, 2018.
- [12] J. Serrà, D. Surís, M. Miron, and A. Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 4548–4557, 2018.
- [13] G. F. Montufar and J. Morton. When does a mixture of products contain a product of mixtures? *SIAM Journal on Discrete Mathematics*, 28(1):321–347, 2014.
- [14] R. K. Srivastava, J. Masci, S. Kazerounian, F. Gomez, and J. Schmidhuber. Compete to compute. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, 2013.
- [15] G. B. Huang, Q. Y. Zhu, and C. K. Siew. Extreme learning machine: Theory and applications. *Neurocomputing*, 70(1–3):489–501, 2006.
- [16] A. Chen, A. Chow, A. Davidson, A. DCunha, A. Ghodsi, S. A. Hong, A. Konwinski, C. Mewald, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, A. Singh, F. Xie, M. Zaharia, R. Zang, J. Zheng, and C. Zumar. Developments in MLflow: A system to accelerate the machine learning lifecycle. In *Proceedings of the Workshop on Data Management for End-to-End Machine Learning (DEEM)*, pages 1–4, 2020.
- [17] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [18] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [19] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.